

BASIC COMPUTING LAB

A Practical, Command-by-Command Lab Manual for UNIX / Linux

Expanded Edition — Five Solved Programs for Every Command

Includes a beginner's Getting Started chapter: history, installation, first login and error handling

Course Title	Basic Computing Lab
Contact Hours / Week (L-T-P)	0 - 0 - 2
Credit	1
Modules	4
Total Experiments	26
Total Solved Programs	130 (5 per experiment)
Prerequisite Chapter	Module 0 — Getting Started (8 sections)
Course Outcomes Covered	CO-1 to CO-6

"The command line is not something to fear — it is the fastest way to talk to a computer."

Program	B. Tech. in CSE-DS	Year, Semester	1st Yr., 1st Sem.
Course Title	Basic Computing Lab	Contact Hours/Week (L-T-P)	0 - 0 - 2
Credit	1	Total Contact Hours	24 (4 modules × 6 hours)

Course Objectives

Enable the student to:

- Introduce students to the UNIX/Linux environment and familiarize them with fundamental system operations, commands and file management techniques.
- Develop proficiency in shell scripting and command-line utilities for automating tasks, managing processes and handling files efficiently.
- Provide hands-on experience with GitHub operations and debugging techniques while enhancing the ability to work with text processing tools, redirection and file compression in a UNIX/Linux environment.

Course Outcomes

On completion of the course, the student will be able to:

CO	Outcome Statement	K-Level	Covered by
CO-1	Be familiar with the UNIX/Linux operating system.	K2	Exp. 1 – 6
CO-2	Develop proficiency in using shell commands and writing basic shell scripts.	K3	Exp. 17 – 20
CO-3	Understand file systems, process management and user permissions.	K2	Exp. 7 – 12
CO-4	Understand basic GitHub operations and debugging of programs.	K3	Exp. 25 – 26
CO-5	Apply fundamental text processing tools and commands such as grep, find and text editors (vi/nano) for efficient file manipulation and searching.	K4	Exp. 13 – 16
CO-6	Utilize redirection, piping and file compression techniques to manage data effectively in a UNIX/Linux environment.	K4	Exp. 21 – 24

K-Level legend (Bloom's Revised Taxonomy): K1 Remember · K2 Understand · K3 Apply · K4 Analyse · K5 Evaluate · K6 Create. This manual targets K2-K4: each experiment first explains *why* a command behaves as it does (K2), then applies it in five graded programs (K3), and closes with viva points that ask the student to compare and justify (K4).

Course Content

Module	Topics	Hours
MODULE 1 Introduction to UNIX/Linux and Basic Commands	Overview of UNIX/Linux operating systems; logging into UNIX/Linux systems; basic system commands: ls, cd, pwd, cp, mv, rm, clear, man, who, date, cal, etc.; understanding the file system hierarchy: /, /home, /bin, /usr, /var, etc.	6 Hours
MODULE 2 File and Process Management	File and directory management — creating, removing and organizing files and directories: mkdir, rmdir, touch, chmod, chown, rm, find, etc.; understanding file permissions and ownership (rwx permissions, chmod command); process management — viewing active processes (ps, top, htop), controlling processes: kill, bg, fg, jobs, nice and renice; understanding process states: running, sleeping, zombie.	6 Hours
MODULE 3 Text Processing and Basic Shell Scripting	Text editors (vi, nano) — creating, editing, saving and exiting files; working with commands like grep, cat, more, less, sed and awk; basic shell scripting — writing simple shell scripts (bash), understanding variables, loops (for, while) and conditional statements (if, elif, else); creating automation scripts for file operations and system monitoring.	6 Hours
MODULE 4 Redirection, Piping and File Compression	Redirection and piping — input/output redirection (>, >>, <), piping () for command chaining; file compression and archiving — working with gzip, tar, zip, unzip; creating and extracting archives for data backup; version control with Git/GitHub.	6 Hours

Mapping of this manual to the syllabus

1. All **26 experiments** of the official teaching plan appear in the same order, with the same CO and K-Level tags printed in the header of every experiment.
2. Each experiment is expanded into **five graded programs** (130 in total), each with Aim, commands, expected output and a bullet-wise explanation.
3. The Table of Contents on the next page doubles as the **Teaching Plan**: it lists Exp. No., topic, CO and K-Level exactly as given in the syllabus.

Table of Contents / Teaching Plan

Module	Exp.	Topics to be Covered	CO	K-Level	Programs
MODULE 0 Getting Started (prerequisite)	0.1 - 0.8	History of UNIX/Linux; why Linux; kernel, shell, terminal and distribution; downloading Ubuntu; setting up the environment (VirtualBox, WSL, live USB, dual boot); starting up and logging in; common errors and how to handle them; terminal survival keys	—	—	Read first
MODULE 1 Introduction to UNIX/Linux & Basic Commands	Exp 1	Log into a UNIX/Linux system and explore basic system information commands: who, date, cal	CO-1	K2	1.1 - 1.5
	Exp 2	Practice navigation commands with options: pwd, ls (-l, -a, -lt), cd	CO-1	K2	2.1 - 2.5
	Exp 3	Perform basic file operations using cp, mv, rm and clear commands	CO-1	K2	3.1 - 3.5
	Exp 4	Explore the file system hierarchy (/ , /home, /bin, /usr, /var) using the man command	CO-1	K2	4.1 - 4.5
	Exp 5	Write and execute a sequence of commands to display system and user information	CO-1	K2	5.1 - 5.5
	Exp 6	Combine basic commands with multiple options for composite file-listing tasks	CO-1	K2	6.1 - 6.5
MODULE 2 File and Process Management	Exp 7	Create, remove and organize directories and files using mkdir, rmdir and touch	CO-3	K2	7.1 - 7.5
	Exp 8	Set and modify permissions and ownership using chmod and chown; interpret rwx	CO-3	K2	8.1 - 8.5
	Exp 9	Search for files and directories using the find command with various options	CO-3	K2	9.1 - 9.5
	Exp 10	Monitor active processes using ps, top and htop	CO-3	K2	10.1 - 10.5
	Exp 11	Control running processes using kill, bg, fg and jobs	CO-3	K2	11.1 - 11.5
	Exp 12	Adjust priority using nice and renice; identify process states (running, sleeping, zombie)	CO-3	K2	12.1 - 12.5
MODULE 3 Text Processing and Basic Shell Scripting	Exp 13	Create, edit and save a text file using the vi editor	CO-5	K4	13.1 - 13.5
	Exp 14	Create, edit and save a text file using the nano editor	CO-5	K4	14.1 - 14.5
	Exp 15	Perform text searching and viewing operations using grep, cat, more and less	CO-5	K4	15.1 - 15.5
	Exp 16	Perform pattern-based text processing using sed and awk	CO-5	K4	16.1 - 16.5
	Exp 17	Write a shell script to declare variables and perform basic input/output operations	CO-2	K3	17.1 - 17.5
	Exp 18	Write a shell script using a for loop to automate a repetitive file operation	CO-2	K3	18.1 - 18.5
	Exp 19	Write a shell script using a while loop and if-elif-else conditional statements	CO-2	K3	19.1 - 19.5
	Exp 20	Write a shell script to automate system monitoring and log the output to a file	CO-2	K3	20.1 - 20.5
MODULE 4 Redirection, Piping, File Compression and Version Control	Exp 21	Demonstrate input/output redirection using >, >> and < operators	CO-6	K4	21.1 - 21.5
	Exp 22	Demonstrate command chaining using the pipe () operator with multiple commands	CO-6	K4	22.1 - 22.5
	Exp 23	Create and extract compressed archives using gzip and tar	CO-6	K4	23.1 - 23.5
	Exp 24	Create and extract compressed archives using zip and unzip for data backup	CO-6	K4	24.1 - 24.5
	Exp 25	Create a GitHub repository and practice the clone operation to set up a local copy	CO-4	K3	25.1 - 25.5
	Exp 26	Practice the GitHub add, commit, push, pull workflow and basic debugging	CO-4	K3	26.1 - 26.5

How to Use This Manual

Part	What it gives you
Objective	The skill being practised, mapped to the Course Outcome (CO) and K-Level of the syllabus.
Concept	A short, plain-language explanation — just enough theory to understand <i>why</i> the command behaves as it does.
General Syntax	The command pattern, with optional parts shown in square brackets.
In simple words	A one-sentence, jargon-free statement of what the program does, for students seeing a terminal for the first time.
Programs 1-5	Five graded programs per experiment. Each one gives the Aim , the exact commands to type (every line carries a plain-English # comment explaining it), the expected output , and a bullet-wise explanation of every line.
Viva-Voce Points	The three questions most often asked on that experiment, with the model answer in brackets.

✓ **NOTE:** All commands are demonstrated on a standard Ubuntu/Debian Linux terminal. The dollar sign \$ at the start of a line is the shell prompt printed by the system — do not type it yourself. Lines beginning with # inside a command block are comments.

00

Getting Started: History, Setup and Your First Login

Before the first experiment: where UNIX and Linux came from, why they are worth learning, how to download and install Ubuntu, how to log in, how to open a terminal, and what to do when something goes wrong. Written for a student who has never seen a command line.

[Sections 0.1 - 0.8](#) | [Read before Experiment 1](#) | [Prerequisite chapter](#)

0.1 A Short History of UNIX and Linux

Purpose: understand what you are learning and where it came from.

Where it all began

In **1969**, two engineers at Bell Laboratories in America — **Ken Thompson** and **Dennis Ritchie** — wrote a small operating system on a spare computer. They called it **UNIX**. It was tiny, but it introduced three ideas that every computer you touch today still uses: files arranged in a tree of folders, many users sharing one machine, and small programs that can be joined together to do a big job.

In **1973** they rewrote UNIX in the **C** language — the same C you are learning this semester. That single decision changed computing, because a system written in C could be moved to a different machine by recompiling it instead of rewriting it from scratch. UNIX spread to universities, and each one added something of its own.

And where Linux came from

By the 1980s UNIX had become commercial and expensive. In **1983** **Richard Stallman** started the **GNU project** to build a free replacement, and wrote most of the tools you will use in this lab — but not the core, called the **kernel**. In **1991**, a 21-year-old Finnish student named **Linus Torvalds** wrote exactly that missing kernel as a hobby project and posted it on the internet with the now-famous message, *"just a hobby, won't be big and professional like GNU."*

The GNU tools plus the Linux kernel made a complete free operating system. That is why the correct full name is **GNU/Linux**, and why Linux is not a *copy* of UNIX — it was written from scratch to behave like UNIX. Today that "hobby" runs the majority of the world's web servers, all 500 of the world's fastest supercomputers, and — inside Android — most of the phones on earth.

Year	Event	Why it matters to you
1969	Thompson and Ritchie write UNIX at Bell Labs	The folder tree and the shell are born
1973	UNIX rewritten in the C language	The system becomes portable between machines
1978	Berkeley releases BSD UNIX	Universities begin improving it freely
1983	Stallman starts the GNU project	The free tools: bash, gcc, grep, tar
1991	Linus Torvalds releases the Linux kernel	The missing piece; the system becomes complete
1992	Linux released under the GPL licence	Anyone may use, study and modify it, free of cost
1993-2004	Debian, Red Hat, SUSE, Ubuntu appear	Ready-made versions you can simply install
2008	Android launches on the Linux kernel	You already carry Linux in your pocket
Today	Servers, cloud, supercomputers, cars, routers	Almost every IT job touches Linux somewhere

✓ **Point to remember:** **UNIX** is the original family of operating systems from 1969. **Linux** is a free system written in 1991 that follows the same rules. The commands in this manual work on both, which is why the course is called UNIX/Linux.

0.2 Why UNIX / Linux? (and how it differs from Windows)

Purpose: know why this subject is in your syllabus.

You already know Windows. So the fair question on day one is: why spend a semester learning a black screen with a blinking cursor? Five honest reasons:

- **It is free and open.** You can install it on any number of machines, look at the source code of every command, and change it. Nothing in this manual requires you to buy a licence.
- **It runs the industry.** Web servers, cloud platforms (AWS, Azure, Google Cloud), Docker, Kubernetes, most databases and every supercomputer run on Linux. Your first internship will almost certainly hand you a Linux terminal.
- **It is stable and safe.** A Linux server routinely runs for years without restarting, and its permission system means an ordinary user cannot damage the system by accident.
- **The command line is faster than clicking.** Renaming 500 files takes one line here and about an hour with a mouse. You will prove this yourself in Experiment 18.
- **It teaches you how computers actually work.** Files, processes, permissions, memory and input/output are all visible and touchable, instead of being hidden behind a window.

Windows and Linux side by side

Idea	Windows	Linux
Where files start	Several drives: C:, D:, E:	One tree that starts at /
Folder separator	Backslash: C:\Users\Rahul	Forward slash: /home/rahul
Your personal folder	C:\Users\YourName	/home/yourname (short form: ~)
Case of names	file.txt = FILE.TXT	file.txt is different from FILE.TXT
Main way to work	Mouse and windows	Typed commands in a terminal
Installing software	Download a .exe and click Next	One command: sudo apt install name
Administrator	Run as Administrator	sudo before the command
File extensions	Decide the file type	Do not matter; permissions decide

⚠ **The one that catches everybody:** Linux is **case sensitive**. Notes.txt , notes.txt and NOTES.txt are three different files, and LS is not the same as ls . When a command "does not work" in your first week, a capital letter is the reason about half the time.

0.3 Kernel, Shell, Terminal and Distribution

Purpose: learn the four words used on every page of this manual.

Four words are used constantly in this manual. Learn them now and the rest of the course becomes much easier.

Word	What it actually is	Everyday comparison
Kernel	The core program that talks to the hardware and shares the CPU, memory and disk between everything else. Linux <i>is</i> the kernel.	The engine of a car
Shell	The program that reads what you type, understands it, and asks the kernel to do it. The usual one is called bash .	The driver who works the engine
Terminal	The window in which the shell runs and shows text. Also called the console or command line.	The dashboard you look at
Distribution	The kernel plus the tools plus an installer, packaged so you can actually install it. Ubuntu, Fedora and Debian are distributions.	The complete car you buy

How a command travels through the system

```
You type:  ls -l
           |
           v
    TERMINAL ..... shows the text on screen
           |
           v
    SHELL (bash) ... reads the line, finds the program /bin/ls, checks the options
           |
           v
    KERNEL ..... actually reads the disk and checks your permissions
           |
           v
    HARDWARE ..... the disk returns the data, which travels all the way back up
```

Which distribution does this manual use?

Ubuntu, because it is the friendliest for a first-year student, has the largest amount of help available online, and is what most college labs install. Everything here also works on Debian, Linux Mint and Pop!_OS without any change, and works on Fedora or CentOS with one difference: they use `dnf install` where Ubuntu uses `sudo apt install`.

✓ **LTS means Long Term Support.** Always download the version marked **LTS** (for example **Ubuntu 24.04 LTS**). It receives security updates for five years, while the in-between releases are supported for only nine months.

0.4 Downloading Ubuntu Linux

Purpose: get the installation file and check your machine can run it.

You need one file: a **.iso**, which is a disk image of the whole operating system. It is about **5 GB**, so use a good connection and expect it to take a while.

Step 1 · Download the ISO file

- Open **<https://ubuntu.com/download/desktop>** in any browser.
- Choose the version marked **LTS** (for example *Ubuntu 24.04.1 LTS*) and click **Download**.
- You will get a file named something like `ubuntu-24.04.1-desktop-amd64.iso`.
- Keep it somewhere you can find again, such as your Downloads folder. Do **not** rename it.

Step 2 · Check that the download is not corrupt (optional but wise)

A half-downloaded ISO will fail during installation with a confusing error. Ubuntu publishes a checksum so you can prove the file is complete.

```
In PowerShell on Windows:  
Get-FileHash ubuntu-24.04.1-desktop-amd64.iso -Algorithm SHA256
```

```
On Linux or macOS:  
sha256sum ubuntu-24.04.1-desktop-amd64.iso
```

Compare the long code it prints with the one on the **SHA256SUMS** file on Ubuntu's website. If they match, the download is perfect. If not, download it again.

Check your machine can run it

Requirement	Bare minimum	Comfortable
RAM (memory)	4 GB total	8 GB or more
Free disk space	25 GB	50 GB or more
Processor	2-core, 64-bit	4-core
For a virtual machine	Virtualisation enabled in BIOS	Same, plus 8 GB RAM in the host

⚠ **If VirtualBox later says VT-x is disabled:** virtualisation is switched off in your BIOS. Restart the laptop, press `F2`, `F10` or `Del` during startup, find **Intel VT-x** or **AMD-V** (usually under Advanced or Security) and set it to **Enabled**, then save and exit.

0.5 Setting Up the Environment (four methods)

Purpose: get a working Linux on your own laptop, safely.

There are four ways to get a Linux terminal in front of you. Read the table, pick one, then follow that section only.

Method	Best for	Risk to your Windows
A. VirtualBox (virtual machine)	Almost everyone. Linux runs in a window inside Windows.	None — recommended
B. WSL (Windows Subsystem for Linux)	Windows 10/11 users who only need the terminal.	None
C. Live USB	Trying Linux without installing anything.	None, but nothing is saved
D. Dual boot	Full speed, permanent use.	High — can erase Windows if done wrong

△ **Advice for first semester:** use **method A or B**. Do not attempt dual boot on your only laptop until you are comfortable with Linux and have a backup of your data.

Method A — Install Ubuntu inside VirtualBox (recommended)

A1 · Install VirtualBox

- Download it from <https://www.virtualbox.org/wiki/Downloads> and choose *Windows hosts*.
- Run the installer and click Next through every screen. Your network will blink off for a second — that is normal.
- Open VirtualBox when it finishes.

A2 · Create the virtual machine

- Click **New**. Name: *Ubuntu-Lab*. Type: *Linux*. Version: *Ubuntu (64-bit)*.
- If only 32-bit appears in the list, virtualisation is disabled in your BIOS — see the warning in section 0.4.
- Memory: **4096 MB** if your laptop has 8 GB, or 2048 MB if it has only 4 GB.
- Processors: **2**. Hard disk: **Create a virtual hard disk now, 25 GB**, dynamically allocated.
- Select the new machine, click **Settings** → **Storage**, click the empty CD icon, then the small disc icon on the right, and choose your downloaded **.iso** file.

A3 · Install Ubuntu inside it

- Click **Start**. The machine boots from the ISO and shows a purple screen.
- Choose **Install Ubuntu** (not *Try Ubuntu*), then pick your language and keyboard.
- Choose **Normal installation** and tick *Download updates while installing* if you have internet.
- At *Installation type* choose **Erase disk and install Ubuntu**. This erases only the 25 GB virtual disk, **not** your real Windows disk — it cannot see it.
- Set your name, a computer name, a **username** and a **password**. Write them down. The username must be lowercase with no spaces, for example *student*.
- Wait 15–25 minutes, then click **Restart Now**. If it asks you to remove the installation medium, just press `Enter`.

Method B — Windows Subsystem for Linux (fastest to set up)

WSL gives you a real Ubuntu terminal inside Windows with no virtual machine at all. Open **PowerShell as Administrator** (right-click the Start button → *Terminal (Admin)*) and type:

```
wsl --install
wsl --install -d Ubuntu-24.04      # ask for a particular version
wsl --list --verbose              # check what is installed
wsl                               # start Ubuntu
```

OUTPUT

```
Installing: Ubuntu
Ubuntu has been installed.
Please create a default UNIX user account.
Enter new UNIX username: student
New password:
Restart your computer to finish installation.
```

- Restart Windows when it asks, then search for **Ubuntu** in the Start menu and open it.
- The first start takes a minute while it unpacks, then asks for a username and password.
- Your Windows drives appear inside Linux at `/mnt/c`, so `cd /mnt/c/Users` reaches your Windows files.
- Limitation:** WSL has no graphical desktop by default, so Experiment 10's `htop` works but a full desktop does not. Everything else in this manual works normally.

Method C — Live USB (try without installing)

- Download **Rufus** from <https://rufus.ie> and insert an 8 GB pen drive (it will be erased).
- In Rufus choose your pen drive, click **SELECT** and pick the Ubuntu ISO, leave the other settings alone, and click **START**.
- Restart the laptop and press the boot-menu key repeatedly — usually `F12`, `F9`, `F10` or `Esc` — then choose the USB drive.
- Choose **Try Ubuntu**. You now have a full working Linux that changes nothing on your disk.
- Remember:** everything you create disappears when you shut down. Good for one lab session, not for your records.

Method D — Dual boot (only with a teacher present)

- **Back up your data first.** A mistake here erases Windows.
- In Windows, open *Disk Management* and **shrink** the C: drive to leave at least 50 GB of unallocated space.
- Turn off **Fast Startup** in Windows power settings, and turn off **Secure Boot** in the BIOS.
- Boot the live USB, choose **Install Ubuntu**, and at *Installation type* select **Install alongside Windows Boot Manager** — never *Erase disk*.
- After installing, a menu called **GRUB** appears at every startup letting you choose Ubuntu or Windows.

0.6 Starting Up, Logging In and Opening the Terminal

Purpose: reach a working prompt and type your first commands.

Logging in

Linux asks who you are before it lets you do anything. This is not a formality: your username decides which files you may read and what you may change.

Step 1 · The login screen

- Click your username, type the **password** you chose during installation, press `Enter`.
- Nothing appears as you type the password** — no dots, no stars. That is deliberate, not a broken keyboard. Type it and press Enter.
- On a text-only login you will see a prompt like `lab-pc-01 login:` . Type the username, press Enter, then type the password.

Step 2 · Open the terminal

- Press `Ctrl` + `Alt` + `T` — the fastest way, worth memorising today.
- Or click *Activities* in the top-left corner, type **terminal**, and press Enter.
- Or right-click on the desktop and choose *Open in Terminal*.

Step 3 · Read the prompt before you type anything

```
student@lab-pc-01:~$
|      |      |      |
|      |      | +--- $ means an ordinary user  (# would mean root/administrator)
|      |      | +--- ~ is the folder you are in  (~ means your home folder)
|      +----- the name of this computer
+----- your username
```

Everything in this manual is typed **after** the `$`. The `$` is printed by the computer — never type it yourself.

Step 4 · Your first four commands

```
$ whoami           # who am I logged in as?
$ pwd              # which folder am I standing in?
$ ls               # what is in this folder?
$ date             # what is today's date and time?
```

OUTPUT

```
student
/home/student
Desktop  Documents  Downloads  Music  Pictures  Videos
Sun Aug 30 09:16:42 IST 2026
```

If all four worked, your environment is set up correctly and you are ready for Experiment 1.

Step 5 · Update the system once, before the first lab

```
$ sudo apt update      # refresh the list of available software
$ sudo apt upgrade -y   # install the updates that are waiting
$ sudo apt install htop tree zip unzip -y # tools this manual uses
```

OUTPUT

```
[sudo] password for student:
Reading package lists... Done
The following NEW packages will be installed:
  htop tree zip unzip
Setting up htop (3.2.2-2) ...
```

`sudo` means *do this as the administrator*. It asks for **your own** password, and again, nothing appears on screen while you type it.

Step 6 · Leaving properly

```
$ exit              # close the terminal window
$ logout            # end a text-mode login session
$ sudo poweroff     # shut the machine down
$ sudo reboot       # restart the machine
```

In a virtual machine, always shut down from inside Ubuntu rather than closing the VirtualBox window, otherwise files you were editing can be damaged.

0.7 Common Errors and How to Handle Them

Purpose: read an error message and fix it yourself.

You will meet these messages in your first week. None of them means the computer is broken. Read the message — Linux almost always states the exact problem.

Message you see	What it really means	How to fix it
bash: lss: command not found	The shell searched everywhere and found no program with that name. Usually a spelling mistake.	Check the spelling and the capital letters. Type the first two letters and press <code>Tab</code> to let the shell complete it.
Permission denied	Your account is not allowed to do this.	For a script, run <code>chmod +x script.sh</code> first. For a system file, put <code>sudo</code> in front of the command.
No such file or directory	The file or folder you named does not exist here.	Run <code>ls</code> to see the real names, and <code>pwd</code> to confirm which folder you are in.
Is a directory	You used a file command on a folder.	Add <code>-r</code> : <code>cp -r</code> or <code>rm -r</code> .
Directory not empty	<code>rmdir</code> only deletes empty folders.	Empty it first, or use <code>rm -r foldername</code> .
bash: cd: too many arguments	The folder name contains a space, so bash saw two names.	Quote it: <code>cd "My Folder"</code> .
sudo: command not found or user is not in the sudoers file	Your account has no administrator rights.	Use the account you created during installation, or ask the lab administrator.
Nothing appears while typing a password	Not an error — Linux hides password characters on purpose.	Type it anyway and press <code>Enter</code> .
The terminal is stuck and will not return the prompt	A program is still running in the foreground.	Press <code>Ctrl</code> + <code>C</code> to stop it, or <code>Ctrl</code> + <code>Z</code> to pause it (Experiment 11).
You are trapped inside vi	vi is waiting in Normal mode.	Press <code>Esc</code> , then type <code>:q!</code> and press <code>Enter</code> (Experiment 13).
E: Could not get lock /var/lib/dpkg/lock	Another install or the automatic updater is already running.	Wait two minutes and try again.
Temporary failure resolving 'archive.ubuntu.com'	The machine has no working internet connection.	Check the network, then run <code>ping -c 3 8.8.8.8</code> to test it.

0.8 Terminal Survival Keys and Readiness Checklist

Purpose: build the habits that make the terminal fast.

These are not exam topics. They are the habits that make the difference between fighting the terminal and enjoying it. Practise them today.

Key	What it does	Why you will love it
<code>Tab</code>	Completes a half-typed file or command name	Removes spelling mistakes entirely. Type <code>cd Doc</code> then <code>Tab</code> .
<code>Tab Tab</code>	Shows every possible completion	Reminds you of a name you half remember
<code>↑ / ↓</code>	Walks through the commands you already typed	Never retype a long command
<code>Ctrl + C</code>	Stops the running command	Your escape key when something hangs
<code>Ctrl + L</code>	Clears the screen	Same as <code>clear</code> but instant
<code>Ctrl + D</code>	Ends input, or closes the terminal	Ends <code>cat > file</code> neatly
<code>Ctrl + A</code> / <code>Ctrl + E</code>	Jump to start / end of the line	Faster than holding the arrow key
<code>Ctrl + R</code>	Search backwards through your history	Find that long command from yesterday
<code>Ctrl + Shift + C</code> / <code>V</code>	Copy and paste in the terminal	Plain <code>Ctrl + C</code> would stop the command instead
<code>history</code>	Lists everything you have typed	Great for writing up your lab record
<code>man command</code>	Opens the built-in manual	The correct answer to almost any question
<code>command --help</code>	Prints a short summary of the options	Quicker than the full manual

Before you start Experiment 1, check all six:

1. Ubuntu starts and you can log in with your username and password.
2. `Ctrl + Alt + T` opens a terminal.
3. `whoami` prints your username and `pwd` prints `/home/yourname`.
4. `sudo apt update` runs without a network error.
5. You know that `Ctrl + C` stops a stuck command.
6. You know that `Esc` then `:q!` escapes from vi.

01

Introduction to UNIX/Linux and Basic Commands

Get comfortable inside the terminal: log in, look around, move between folders, and perform your first file operations. Every experiment now carries five graded programs with full output and line-by-line explanation.

Experiments 1 - 6 | Duration: 6 Hours | CO-1 | 30 Programs

Objective: Log into a UNIX/Linux system and explore basic system information commands: **who**, **date** and **cal**.

Concept

Every Linux session begins with authentication. Once the shell (bash) starts, it displays a prompt `user@host:~$` and waits for commands. The three foundation commands report *who* is on the machine, *when* it is, and *what* the calendar looks like. These commands need no arguments, which makes them the safest first commands for a beginner.

General Syntax

```
who [options]
date [+FORMAT]
cal [month] [year]
```

Useful format specifiers for date

Specifier	Meaning
%d	Day of month (01-31)
%m	Month number (01-12)
%Y	Four-digit year
%B	Full month name
%A	Full weekday name
%H:%M:%S	Hour : Minute : Second

Practical Programs

Program 1.1 Display all users currently logged into the system

Aim: Use **who** to list every active login session with terminal and login time.

In simple words: Ask the computer: who is using this machine at the moment?

```
$ who      # show every user logged in right now
$ who -H   # same list, but print a heading row on top
$ who am i # show only MY own login session
$ who -q   # quick list of names plus a total count
```

OUTPUT

```
root    tty1      2026-08-30 09:12
student pts/0      2026-08-30 09:15 (192.168.1.14)

NAME    LINE      TIME          COMMENT
root    tty1      2026-08-30 09:12
student pts/0      2026-08-30 09:15 (192.168.1.14)

student pts/0      2026-08-30 09:15

root student
# users=2
```

EXPLANATION

- Column 1 is the **username**, column 2 is the **terminal line** — `tty1` is a physical console, `pts/0` is a pseudo-terminal (a remote or GUI terminal window).
- Column 3 is the **login time**; the bracketed value is the remote IP address of the client machine.
- `-H` prints a header row, `who am i` shows only your own session, and `-q` prints a quick count of users.

Program 1.2 Print the system date and time in different formats

Aim: Use `date` with format specifiers to control exactly how the timestamp is printed.

In simple words: Ask the computer for today's date, and make it print the date the way you want.

```
$ date                # show today's date and the current time
$ date +"%d-%m-%Y"    # print the date as day-month-year
$ date +"%A, %d %B %Y" # print the day name and full month name
$ date +"Time is %H:%M:%S" # print only the time, with my own wording
$ date +"%d/%m/%y %I:%M %p" # print a short date with a 12-hour clock
```

OUTPUT

```
Sun Aug 30 09:16:42 IST 2026
30-08-2026
Sunday, 30 August 2026
Time is 09:16:42
30/08/26 09:16 AM
```

EXPLANATION

- With no argument `date` prints the default full form: weekday, month, day, time, timezone, year.
- A `+` sign introduces a **format string**. Anything that is not a `%` specifier (like the dashes or the word *Time is*) is printed literally, so the whole string must be quoted.
- `%I` gives the 12-hour clock and `%p` adds AM/PM, whereas `%H` gives the 24-hour clock — a very common one-mark question in the viva.

Program 1.3 Display the calendar of a month and of a full year

Aim: Use `cal` to display the current month, a specific month/year, and a whole year.

In simple words: Put a calendar on the screen for any month or any year you choose.

```
$ cal                # show the calendar of the current month
$ cal 3 2027         # show the calendar for that month and year
$ cal 2027 | head -12 # show the calendar for that month and year
```

OUTPUT

```
      August 2026
Su Mo Tu We Th Fr Sa
                1
 2  3  4  5  6  7  8
 9 10 11 12 13 14 15
16 17 18 19 20 21 22
23 24 25 26 27 28 29
30 31

      March 2027
Su Mo Tu We Th Fr Sa
  1  2  3  4  5  6
 7  8  9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30 31
```

EXPLANATION

- With no argument `cal` shows the current month and highlights today's date.
- With **two** arguments the order is always **month first, then year** — `cal 3 2027` is March 2027, while `cal 2027` alone prints all twelve months of that year.
- Because a full year is longer than the screen, it is piped into `head -12` to show only the first twelve lines — an early preview of pipes (Experiment 22).

Program 1.4 Display user identity and machine identity

Aim: Use **whoami**, **id**, **hostname** and **uname** to identify the user and the machine.

In simple words: Ask the computer two questions: who am I, and what machine am I sitting at?

```
$ whoami      # print my username
$ id          # print my user number, group number and groups
$ hostname    # print the name of this computer
$ uname       # print the name of the operating system kernel
$ uname -a    # print ALL system details in one line
```

OUTPUT

```
student
uid=1000(student) gid=1000(student) groups=1000(student),27(sudo)
lab-pc-01
Linux
Linux lab-pc-01 6.8.0-45-generic #45-Ubuntu SMP x86_64 GNU/Linux
```

EXPLANATION

- **whoami** prints only the effective username; **id** prints the numeric **UID**, the primary **GID** and every supplementary group. UID 0 always means *root*.
- **hostname** prints the network name of the machine — useful when you are logged into several lab PCs at once.
- **uname** alone prints only the kernel name; **-a** expands it to kernel name, host, kernel release, build stamp and hardware architecture.

Program 1.5 Generate a one-line login report using command substitution

Aim: Combine **whoami**, **hostname** and **date** into a single formatted line.

In simple words: Join several answers together so the computer prints one neat summary line.

```
$ uptime          # show how long this machine has been switched on
$ who -b          # show the date and time the machine last started
$ echo "User: $(whoami) | Host: $(hostname) | Login date: $(date +%d-%m-%Y)"# print this text on the screen
```

OUTPUT

```
09:24:11 up 1:24,  2 users,  load average: 0.10, 0.05, 0.01
      system boot  2026-08-30 08:00
User: student | Host: lab-pc-01 | Login date: 30-08-2026
```

EXPLANATION

- **uptime** reports how long the machine has been running, the number of logged-in users, and the 1/5/15-minute CPU load averages.
- **who -b** prints the last system boot time, which should agree with the *up* value of **uptime**.
- **Command substitution** **\$(command)** runs the inner command first and pastes its output into the string. This is the single most important idea in this experiment because every later script uses it.

✓ **NOTE:** The **\$** at the start of a line is the shell prompt printed by the system — never type it yourself.

Viva-Voce / Exam Points

1. What is the difference between **who** and **whoami**? (**who** lists all sessions; **whoami** prints only your username.)
2. In **cal 3 2027**, which number is the month? (The first one — month always precedes year.)
3. What does UID 0 signify? (The *root* / superuser account.)

Objective: Practise navigation commands with options: **pwd**, **ls** (-l, -a, -lt) and **cd**.

Concept

Linux stores everything in a single inverted tree that starts at the root directory `/`. At any moment your shell sits in one node of that tree, called the **current working directory**. `pwd` tells you where you are, `cd` moves you, and `ls` shows what is inside. A path that starts with `/` is **absolute**; anything else is **relative** to where you currently stand.

General Syntax

```
pwd
cd [directory]
ls [options] [path]
```

ls flag reference

Flag	Meaning
-l	Long listing: permissions, links, owner, group, size, date, name
-a	Show hidden files (names beginning with a dot)
-t	Sort by modification time, newest first
-r	Reverse the current sort order
-h	Human-readable sizes (K, M, G)
-R	Recurse into every sub-directory

Practical Programs

Program 2.1 Print the current working directory

Aim: Use **pwd** to print the absolute path of the directory you are standing in.

In simple words: Every folder has an address. This command prints the address you are standing at.

```
$ pwd          # print which folder I am standing in
$ cd /usr/share # move into that folder
$ pwd          # print which folder I am standing in
$ cd ~         # go back to my home folder
$ pwd          # print which folder I am standing in
```

OUTPUT

```
/home/student
/usr/share
/home/student
```

EXPLANATION

- `pwd` always prints an **absolute** path, i.e. the complete route from the root `/` down to your position.
- After `cd /usr/share` the prompt location changes, and `pwd` proves it.
- `cd` with **no argument at all** is a shortcut that returns you to your home directory — the fastest way to recover when you are lost.

Program 2.2 List directory contents in plain and long format

Aim: Compare `ls` with `ls -l` and decode every column of the long listing.

In simple words: See what is inside a folder — first just the names, then all the details.

```
$ ls          # list the files in this folder
$ ls -l       # list with details: permissions, owner, size, date
```

OUTPUT

```
Desktop Documents Downloads lab_work notes.txt

total 20
drwxr-xr-x 2 student student 4096 Aug 28 10:02 Desktop
drwxr-xr-x 2 student student 4096 Aug 28 10:02 Documents
drwxr-xr-x 2 student student 4096 Aug 28 10:03 Downloads
drwxr-xr-x 3 student student 4096 Aug 29 11:40 lab_work
-rw-r--r-- 1 student student 220 Aug 28 09:55 notes.txt
```

EXPLANATION

- Plain `ls` prints names only, in alphabetical order, in columns.
- In the long listing the nine columns are: **type+permissions**, **link count**, **owner**, **group**, **size in bytes**, **month**, **day**, **time**, **name**.
- The very first character is the file **type**: `d` = directory, `-` = ordinary file, `l` = symbolic link. The word *total 20* is the disk blocks used.

Program 2.3 Reveal hidden configuration files

Aim: Use `ls -a` and `ls -la` to display dot-files that are normally invisible.

In simple words: Some files are hidden on purpose. This command makes them visible.

```
$ ls -a          # list everything, including hidden files
$ ls -la | head -6 # list the contents of that folder
```

OUTPUT

```
.  .. .bashrc .profile .ssh Desktop Documents Downloads notes.txt

total 44
drwxr-x--- 6 student student 4096 Aug 30 09:20 .
drwxr-xr-x 3 root     root    4096 Aug 20 12:00 ..
-rw-r--r-- 1 student student 3771 Aug 20 12:00 .bashrc
-rw-r--r-- 1 student student  807 Aug 20 12:00 .profile
drwx----- 2 student student 4096 Aug 25 15:11 .ssh
```

EXPLANATION

- Any file whose name begins with a dot is **hidden** by convention — it is not a security feature, merely a way of keeping configuration files out of the way.
- Two special entries always appear: `.` means *this* directory and `..` means the **parent** directory. They are what make `cd ..` work.
- Flags can be bundled, so `ls -la` is identical to `ls -l -a`.

Program 2.4 Sort a listing by modification time

Aim: Use `ls -lt` and `ls -ltr` to find the newest and the oldest files quickly.

In simple words: Put the newest files at the top so you can see what changed today.

```
$ ls -lt          # long list, newest file first
$ ls -ltr         # long list, oldest file first
$ ls -lt | head -3 # list the contents of that folder
```

OUTPUT

```
total 20
-rw-r--r-- 1 student student  220 Aug 30 09:20 notes.txt
drwxr-xr-x 3 student student 4096 Aug 29 11:40 lab_work
drwxr-xr-x 2 student student 4096 Aug 28 10:02 Desktop

total 20
drwxr-xr-x 2 student student 4096 Aug 28 10:02 Desktop
drwxr-xr-x 3 student student 4096 Aug 29 11:40 lab_work
-rw-r--r-- 1 student student  220 Aug 30 09:20 notes.txt
```

EXPLANATION

- `-t` replaces alphabetical order with **time order, newest first**; adding `-r` reverses it so the oldest appears first — the standard trick for reading log directories.
- The flags combine into one token: `-ltr` means long + time-sorted + reversed.
- Piping into `head -3` keeps only the three most recently modified entries.

Program 2.5 Move around the directory tree with cd

Aim: Practise absolute paths, relative paths and the three navigation shortcuts `..`, `~` and `-`.

In simple words: Walk around the folder tree: go in, go up, go home, and go back.

```
$ cd /home/student/Documents      # absolute path
$ pwd                             # print which folder I am standing in
$ cd ..                          # one level up
$ pwd                             # print which folder I am standing in
$ cd Downloads                    # relative path
$ pwd                             # print which folder I am standing in
$ cd ~                           # home directory
$ pwd                             # print which folder I am standing in
$ cd -                           # previous directory
$ pwd                             # print which folder I am standing in
$ cd ../../etc                   # move into that folder
$ pwd                             # print which folder I am standing in
```

OUTPUT

```
/home/student/Documents
/home/student
/home/student/Downloads
/home/student
/home/student/Downloads
/etc
```

EXPLANATION

- An **absolute** path begins at `/` and works from anywhere; a **relative** path such as `Downloads` only works if that folder exists below your current position.
- `..` climbs one level, and it can be chained: `cd ../../etc` climbs two levels and then descends into `/etc`.
- `~` always expands to your home directory, while `cd -` jumps back to the *previous* directory — running it twice returns you to where you started.

Viva-Voce / Exam Points

1. Which command shows hidden files and why are they hidden? (`ls -a`; any name starting with a dot.)
2. Difference between `cd ~` and `cd -`? (Home directory vs. previous directory.)
3. What does the first character of `drwxr-xr-x` indicate? (It is a directory.)

Objective: Perform basic file operations using **cp**, **mv**, **rm** and **clear**.

Concept

Copying, renaming/moving and deleting are the three actions you will perform most often. Note that Linux has **no separate rename command** — **mv** does both moving and renaming, because renaming is simply moving a file to a new name in the same directory. There is also no Recycle Bin: **rm** is permanent.

General Syntax

```
cp [options] source destination
mv source destination
rm [options] file
clear
```

Important options

Option	Applies to	Effect
-r	cp / rm	Recursive — required for directories
-i	cp / mv / rm	Interactive — ask before overwriting or deleting
-v	cp / mv / rm	Verbose — report every action performed
-f	rm	Force — never prompt, never complain (dangerous)

Practical Programs

Program 3.1 Create a file and make a backup copy

Aim: Create a text file with `echo` and duplicate it using `cp`.

In simple words: Make a small text file, then make a spare copy of it.

```
$ echo "Hello Linux" > file1.txt # print this text on the screen
$ cat file1.txt                 # show the whole contents of the file
$ cp file1.txt file1_backup.txt # make a copy of the file
$ ls                            # list the files in this folder
$ cp -v file1.txt /tmp/         # copy the file and report what was copied
```

OUTPUT

```
Hello Linux
file1.txt file1_backup.txt
'file1.txt' -> '/tmp/file1.txt'
```

EXPLANATION

- `echo "text" > file` writes a line into a new file; the `>` operator is studied fully in Experiment 21.
- `cp source destination` leaves the source untouched and creates an independent duplicate — editing one afterwards does not change the other.
- If the destination is an existing **directory** (like `/tmp/`), the copy keeps its original name inside that directory. `-v` prints what was done.

Program 3.2 Copy an entire directory recursively

Aim: Use `cp -r` to duplicate a folder together with all of its contents.

In simple words: Copy a whole folder, together with everything stored inside it.

```
$ mkdir -p archive/notes # create the folder and any missing parent folders
$ touch archive/a.txt archive/notes/b.txt # create an empty file, or update its date
$ cp archive archive_copy # make a copy of the file
$ cp -r archive archive_copy # copy the folder and everything inside it
$ ls -R archive_copy        # list the contents of that folder
```

OUTPUT

```
cp: -r not specified; omitting directory 'archive'

archive_copy:
a.txt  notes

archive_copy/notes:
b.txt
```

EXPLANATION

- Plain `cp` **refuses** to copy a directory and prints the error shown — this is the single most common beginner mistake in this experiment.
- `-r` (recursive) tells `cp` to walk into the folder and copy every file and sub-folder inside it.
- `ls -R` confirms the copy by listing the new tree level by level.

Program 3.3 Rename a file and move it into a folder

Aim: Show that `mv` performs both renaming and relocation.

In simple words: Give a file a new name, then move it into a different folder.

```
$ ls          # list the files in this folder
$ mv file1_backup.txt renamed.txt      # rename
$ ls          # list the files in this folder
$ mkdir store # create a new folder
$ mv renamed.txt store/                # move
$ ls store   # list what is now inside the store folder
$ mv store/renamed.txt store/final.txt # move + rename in one step
$ ls store   # list what is now inside the store folder
```

OUTPUT

```
file1.txt  file1_backup.txt
file1.txt  renamed.txt
renamed.txt
final.txt
```

EXPLANATION

- When the destination is a **name**, `mv` renames the file; when the destination is an existing **directory**, it moves the file into it.
- Unlike `cp`, `mv` does not duplicate data — the original entry disappears, so it is much faster on large files within the same disk.
- `mv` silently overwrites an existing target, so `mv -i` is recommended while learning.

Program 3.4 Delete files safely and delete directories

Aim: Use `rm`, `rm -i` and `rm -r` and observe the error messages.

In simple words: Delete files safely, and see the errors Linux gives when you get it wrong.

```
$ rm renamed_typo.txt # delete the file permanently
$ rm -i file1.txt     # delete the file, but ask me first
rm: remove regular file 'file1.txt'? y
$ rm archive          # delete the file permanently
$ rm -r archive        # delete the folder and everything in it
$ ls                  # list the files in this folder
```

OUTPUT

```
rm: cannot remove 'renamed_typo.txt': No such file or directory

rm: cannot remove 'archive': Is a directory

archive_copy  store
```

EXPLANATION

- Deleting a non-existent file produces a clear *No such file or directory* error and changes nothing.
- `-i` asks for confirmation before each deletion; answer `y` to proceed or `n` to cancel. Always use it while you are learning.
- Plain `rm` cannot delete a directory — `rm -r` is required, and it removes everything inside without further warning.

Program 3.5 Clear the screen and verify a complete file workflow

Aim: Run a full create - copy - rename - delete cycle and clean the terminal with `clear`.

In simple words: Do the whole life of a file — create, copy, rename, check, delete — then tidy the screen.

```
$ echo "Basic Computing Lab" > lab.txt # print this text on the screen
$ cp lab.txt lab_bkp.txt              # make a copy of the file
$ mv lab_bkp.txt lab_final.txt        # move the file, or rename it
$ ls -l lab*                          # show details of that file or folder
$ rm -i lab_final.txt                 # delete the file, but ask me first
$ ls lab*                            # list every file whose name starts with lab
$ clear                               # wipe the screen clean
```

OUTPUT

```
-rw-r--r-- 1 student student 20 Aug 30 10:02 lab.txt
-rw-r--r-- 1 student student 20 Aug 30 10:02 lab_final.txt
rm: remove regular file 'lab_final.txt'? y
lab.txt
```

EXPLANATION

- `lab*` is a **wildcard**: the shell expands it to every name starting with *lab* before the command even runs.
- The whole life-cycle of a file — create, duplicate, rename, verify, delete — is covered in six lines.
- `clear` only scrolls the visible screen away; the command history is still available with the Up-arrow key, so nothing is actually lost.

⚠ **CAUTION:** `rm` deletes permanently — there is no Recycle Bin in the terminal. Never run `rm -rf /` or `rm -rf *` in a directory you have not verified with `pwd`.

Viva-Voce / Exam Points

1. Why does `cp` need `-r` but `mv` does not? (`mv` only relinks the directory entry; `cp` must duplicate every file inside.)
2. Which command both renames and moves? (`mv`.)

3. What is the safest flag to add to `rm` while learning? `(-i.)`